# Konkurens programozás

## Félbeszakítás, leállítás




**Kozsik Tamás**

ELTE Eötvös Loránd Tudományegyetem

# InterruptedException

- `Thread.join()`
- `Thread.sleep(int)`
- `invokeAll(Collection<? extends Callable<T>>)`
- `ExecutorService.awaitTermination(long,TimeUnit)`
- `Future.get()`
- `Object.wait(int)`



# InterruptedException

- `Thread.join()`
- `Thread.sleep(int)`
- `invokeAll(Collection<? extends Callable<T>>)`
- `ExecutorService.awaitTermination(long,TimeUnit)`
- `Future.get()`
- `Object.wait(int)`

`Thread.interrupt()`



# Masszív számolás

```
import java.util.List;
import java.math.BigInteger;
...

static void collectProbablePrimes( List<BigInteger> list,
                                   BigInteger from,
                                   BigInteger to        ){

    from = from.subtract(BigInteger.ONE).nextProbablePrime();
    while( from.compareTo(to) <= 0 ){
        list.add(from);
        from = from.nextProbablePrime();
    }
}
```



# Aszinkron módon meghívva

```
static void collectProbablePrimes( List<BigInteger> list,
                                   BigInteger from,
                                   BigInteger to        ){
    ...
}
```

```
public static void main( String[] args ) throws InterruptedException {
    ...
    Thread t = new Thread( () -> collectProbablePrimes(pps,a,b) );
    t.start();
    ...
    t.join();
    ...
}
```



# Thread osztályból származtatva

```
class PrimeFinder extends Thread {

    private final List<BigInteger> list;
    private BigInteger from, to;

    PrimeFinder( List<BigInteger> list, BigInteger from, BigInteger to ){
        this.list = list; this.from = from; this.to = to;
    }

    @Override public void run(){
        from = from.subtract(BigInteger.ONE).nextProbablePrime();
        while( from.compareTo(to) <= 0 ){
            list.add(from);
            from = from.nextProbablePrime();
        }
    }

}
```



# Használat

```
public static void main( String[] args ) throws InterruptedException {
    ...
    PrimeFinder t = new PrimeFinder(pps,a,b);
    t.start();
    ...
    t.join();
    ...
}
```



# Szál leállításához

```java
/**
  Thread-safe boolean flag, initialized to false.
*/
public class Flag {
    private boolean value = false;
    public synchronized boolean get(){
        return value;
    }
    public synchronized void flip(){
        value = !value;
    }
}
```



# Leállítható szál

```
class PrimeFinder extends Thread {
    private final List<BigInteger> list;
    private BigInteger from, to;
    private final Flag terminated = new Flag();

    PrimeFinder( ... ){ ... }

    @Override public void run(){
        from = from.subtract(BigInteger.ONE).nextProbablePrime();
        while( !terminated.get() && from.compareTo(to) <= 0 ){
            list.add(from);
            from = from.nextProbablePrime();
        }
    }

    public void terminate(){ terminated.flip(); }
}
```



# Szál leállítása

```
public static void main( String[] args ) throws InterruptedException {
    List<BigInteger> pps = new LinkedList<>();
    BigInteger a = new BigInteger(args[0]), b = new BigInteger(args[1]);
    PrimeFinder t = new PrimeFinder(pps,a,b);
    t.start();
    Thread.sleep(10);
    t.terminate();
    t.join();
    System.out.println(pps);
}
```



# java.util.concurrent.atomic.AtomicBoolean

```
class PrimeFinder extends Thread {
    private final List<BigInteger> list;
    private BigInteger from, to;
    private final AtomicBoolean terminated = new AtomicBoolean(false);

    PrimeFinder( ... ){ ... }

    @Override public void run(){
        from = from.subtract(BigInteger.ONE).nextProbablePrime();
        while( !terminated.get() && from.compareTo(to) <= 0 ){
            list.add(from);
            from = from.nextProbablePrime();
        }
    }

    public void terminate(){ terminated.set(true); }
}
```



# java.util.concurrent.atomic

- AtomicBoolean
- AtomicInteger
- AtomicLong
- AtomicIntegerArray
- AtomicReference<T>
- …

A synchronized-nál hatékonyabban implementálva!



# volatile változóval

```
class PrimeFinder extends Thread {
    private final List<BigInteger> list;
    private BigInteger from, to;
    private volatile boolean terminated = false;

    PrimeFinder( ... ){ ... }

    @Override public void run(){
        from = from.subtract(BigInteger.ONE).nextProbablePrime();
        while( !terminated && from.compareTo(to) <= 0 ){
            list.add(from);
            from = from.nextProbablePrime();
        }
    }

    public void terminate(){ terminated = true; }
}
```



# Thread.interrupted()

```
class PrimeFinder extends Thread {

    private final List<BigInteger> list;
    private BigInteger from, to;

    PrimeFinder( ... ){ ... }

    @Override public void run(){
        from = from.subtract(BigInteger.ONE).nextProbablePrime();
        while( !interrupted() && from.compareTo(to) <= 0 ){
            list.add(from);
            from = from.nextProbablePrime();
        }
    }

}
```



# Thread.interrupt()

```
public static void main( String[] args ) throws InterruptedException {
    ...
    PrimeFinder t = new PrimeFinder(pps,a,b);
    t.start();
    ...
    t.interrupt();
    t.join();
    ...
}
```



# Runnable számítás megszakítása

```
static void collectProbablePrimes( List<BigInteger> list,
                                   BigInteger from,
                                   BigInteger to        ){
    from = from.subtract(BigInteger.ONE).nextProbablePrime();
    while( !Thread.interrupted() && from.compareTo(to) <= 0 ){
        list.add(from);
        from = from.nextProbablePrime();
    }
}

public static void main( String[] args ) throws InterruptedException {
    ...
    Thread t = new Thread( () -> collectProbablePrimes(pps,a,b) );
    t.start();
    ...
    t.interrupt(); t.join();
    ...
}
```



# Thread.interrupted() és Thread.isInterrupted()

```
public static boolean interrupted()
```

Lekérdezi és kinullázza a *flag*et.

```
public boolean isInterrupted()
```

Csak lekérdezi a *flag*et.



# Thread.interrupted() és Thread.isInterrupted()

```
public static boolean interrupted()
```

Lekérdezi és kinullázza a *flag*et.

```
public boolean isInterrupted()
```

Csak lekérdezi a *flag*et.

```
Thread.currentThread().isInterrupted()
```



# Félbeszakítás – miért van rá szükség?

- Felhasználó visszavonja a kérését
- Időkorlátos tevékenység ideje lejár
- Egy másik komponens megoldást talált
- Hiba történt valamelyik komponensben
- Az alkalmazás/szolgáltatás leáll



# Folyamatok félbeszakítása

- Szál: `interrupt`
- Feladat: `cancel`
- ExecutorService: `shutdown` és `shutdownNow`



# Megszakítás

A szálaknak jelezhetünk

- Jelzés: `thread.interrupt()`
- Észlelés: `thread.isInterrupted()`, `Thread.interrupted()`
- Lehetséges reakciók
  - Várakozás megszakítása: `InterruptedException`
    - `Thread.sleep(int)`
    - `thread.join()`
    - `wait()`
    - `BlockingQueue`…
  - Ciklus befejezése



# Példa

```
static void collectProbablePrimes( List<BigInteger> list,
                                   BigInteger from,
                                   BigInteger to        ){
    from = from.subtract(BigInteger.ONE).nextProbablePrime();
    while( !Thread.interrupted() && from.compareTo(to) <= 0 ){
        list.add(from);
        from = from.nextProbablePrime();
    }
}

public static void main( String[] args ) throws InterruptedException {
    ...
    Thread t = new Thread( () -> collectProbablePrimes(pps,a,b) );
    t.start();
    ...
    t.interrupt(); t.join();
    ...
}
```



# Feladat félbeszakítása

## java.util.concurrent.Future

```
boolean cancel(boolean mayInterruptIfRunning)
boolean isCancelled()
...
```

- `cancel(false)`: ne futtasd, ha még nem fut
- `cancel(true)`: ... és szakítsd félbe a szálat, ha fut
  - Standard Executorokban való futtatásnál rendben van a `cancel(true)`
  - Egyébként csak akkor, ha ismerjük a futtató szál reakcióját



# Példa – standard executorban futtatva

```
static BigInteger factorize( BigInteger num ){
    BigInteger i = BigInteger.TWO;
    boolean found = false;
    while( !Thread.interrupted() &&
           num.compareTo( i.pow(2) ) > 0 &&
           !(found = num.remainder(i).equals(BigInteger.ZERO))
         ) i = i.nextProbablePrime();
    return found ? i : null;
}

void caller( ExecutorService es, BigInteger num ) throws Exception {
    Future<BigInteger> factor = es.submit( () -> factorize(num) );
    ...
    if( factor.isDone() ) System.out.println( factor.get() );
    else factor.cancel(true);
}
```



# Standard executorban futtatva, interrupt-státuszt megőrizve

```
static BigInteger factorize( BigInteger num ){
    BigInteger i = BigInteger.TWO;
    boolean found = false;
    while( !Thread.currentThread().isInterrupted() &&
           num.compareTo( i.pow(2) ) > 0 &&
           !(found = num.remainder(i).equals(BigInteger.ZERO))
         ) i = i.nextProbablePrime();
    return found ? i : null;
}

void caller( ExecutorService es, BigInteger num ) throws Exception {
    Future<BigInteger> factor = es.submit( () -> factorize(num) );
    ...
    if( factor.isDone() ) System.out.println( factor.get() );
    else factor.cancel(true);
}
```



# Ismeretlen működésű szálra készülve

```
class Factorizer implements Callable<BigInteger>{
    private final BigInteger num;
    private volatile boolean stopped = false;

    Factorizer( BigInteger num ){ this.num = num; }

    public BigInteger call(){
        BigInteger i = BigInteger.TWO;
        boolean found = false;
        while( !stopped &&
               num.compareTo( i.pow(2) ) > 0 &&
               !(found = num.remainder(i).equals(BigInteger.ZERO))
             ) i = i.nextProbablePrime();
        return found ? i : null;
    }

    public void stop(){ stopped = true; }
}
```



# Ismeretlen működésű szálra készülve

```
class Factorizer implements Callable<BigInteger>{
    private volatile boolean stopped = false;
    ...
    public BigInteger call(){
        ...
        while( !stopped && ...) ...
        ...
    }

    public void stop(){ stopped = true; }
}
```

```
Factorizer task = new Factorizer(num);
Future<BigInteger> factor = es.submit( task );
...
if( factor.isDone() ){ ... factor.get() ... }
else { factor.cancel(false); task.stop(); }
```

# ExecutorService leállítása

## java.util.concurrent.ExecutorService

```
...
void shutdown()
boolean isShutdown()
List<Runnable> shutdownNow()
...
```

## shutdown()

- ne fogadj el több feladatot
- az eddigieket még hajts végre

## shutdownNow()

- ne fogadj el több feladatot
- az eddig el nem kezdetteket már ne is kezd el
- a már elkezdetteket próbáld meg félbeszakítani

# Leállítás, majd leállás bevárása

```
String[] sort( String[] inputs, int parallelism ) throws Exception {
    String[] outputs = new String[inputs.length];
    ExecutorService exec = Executors.newFixedThreadPool(parallelism);
    for( int i=0; i<inputs.length; ++i ){
        final int I = i;
        exec.execute( () -> arrangeOne(I,inputs,outputs) );
    }
    exec.shutdown();
    exec.awaitTermination(1,TimeUnit.SECONDS);
    return outputs;
}

void arrangeOne( int i, String[] inputs, String[] outputs ){ ... }
```

